

COURSE MATERIAL



Module 2: SQL Fundamentals (DDL & DML)

Module 2 - 8 Hours

SQL Fundamentals (DDL & DML)

Module 2 at a glance

Duration	8 hours (4 hours theory, 4 hours lab)
Prerequisites	Module 1, with a loaded <code>icicibank</code> database
You will build	The <code>fixed_deposit</code> table for the ICICI Smart Saver product, with rules and data
You will carry forward	<code>fixed_deposit</code> , used in Modules 3, 5, 6 and 12

Learning outcomes

By the end of this module you can:

1. Choose an appropriate data type for any banking column, and explain why floating point is unsafe for money.
2. Declare and name all six constraint types, and predict which write each one will reject.
3. Write `CREATE TABLE`, `ALTER TABLE` and `DROP TABLE`, and distinguish `DROP`, `DELETE` and `TRUNCATE`.
4. Write `INSERT`, `UPDATE` and `DELETE` safely against production-shaped data.
5. Write `SELECT` with filtering and sorting, and handle `NULL` correctly using three-valued logic.

The business problem

Ticket ICICI-4482 - Launch "ICICI Smart Saver" fixed deposit

Marketing launches Smart Saver on 1 April. A customer books a deposit for a fixed term at a fixed rate and chooses whether it auto-renews at maturity. Data engineering must deliver: a table to hold the deposits, rules that stop bad data getting in, the first twenty bookings from the branch pilot, and a daily "maturing this week" list for the call centre.

1. The two halves of SQL

	DDL - Data Definition Language	DML - Data Manipulation Language
Purpose	Define structure	Work with data
Statements	CREATE, ALTER, DROP, TRUNCATE	INSERT, UPDATE, DELETE, SELECT
Changes	The schema	The rows
Run by	Data engineers and DBAs, occasionally	Applications, constantly

There is also **DCL** (GRANT, REVOKE), covered in Module 9, and **TCL** (COMMIT, ROLLBACK, SAVEPOINT), covered in Module 6.

An important difference between engines. On Oracle and MySQL, DDL causes an **implicit commit**: running a CREATE or ALTER silently commits whatever transaction you had open. On PostgreSQL and SQL Server, DDL is transactional and can be rolled back. Assuming the wrong behaviour during a migration is a common and expensive mistake.

2. Data types

2.1 Numeric types

Type	Range or precision	ICICI Bank use
SMALLINT	-32,768 to 32,767	tenure_months, tenure_days
INTEGER	About +/- 2.1 billion	customer_id, account_id, branch_id
BIGINT	About +/- 9.2 quintillion	txn_id
NUMERIC(p, s) or DECIMAL(p, s)	Exact, p total digits with s after the decimal point	Every money column
REAL, DOUBLE PRECISION, FLOAT	Approximate binary floating point	Never for money

NUMERIC(15,2) stores up to 9,999,999,999,999.99, which comfortably covers any single ICICI Bank balance or transaction.

Why txn_id is BIGINT. At two million transactions a month, INTEGER overflows in about 90 years. That sounds safe until you remember that transaction tables are often loaded with history from a migrated system, and that the same column type gets copied into every downstream table. Banks have had production outages from exactly this.

2.2 Why floating point is banned for money

```
SELECT CAST(0.1 AS REAL) + CAST(0.2 AS REAL) = CAST(0.3 AS REAL);
-- false

SELECT CAST(0.1 AS NUMERIC) + CAST(0.2 AS NUMERIC) = CAST(0.3 AS NUMERIC);
-- true
```

Binary floating point cannot represent 0.1 exactly, in the same way decimal cannot represent one third exactly. Each individual error is tiny. Summed across a day of transactions across a branch network, the bank's cash position no longer balances, and the difference cannot be traced to any single transaction.

NUMERIC and DECIMAL store digits, not binary fractions, so the arithmetic is exact. They are slower. A bank pays that price without discussion.

2.3 Character types

Type	Behaviour	Use for
CHAR(n)	Fixed length, padded with trailing spaces	ifsc_code CHAR(11), currency CHAR(3)
VARCHAR(n)	Variable length up to n	first_name, email, narration
TEXT / CLOB	Unbounded	Complaint text, long notes

Use CHAR only when every value genuinely has the same length. An IFSC code is always 11 characters and a currency code always 3, so CHAR is honest there. A customer name is not, and storing it in CHAR(40) wastes space and introduces trailing-space comparison problems that differ between engines.

Sizing. `VARCHAR(4000)` for a first name is not "flexible", it is a missing decision. The length is documentation, and it is the last line of defence against a broken upstream feed writing an entire XML document into a name column.

2.4 Date and time types

Type	Stores	ICICI Bank column
DATE	A calendar date. Oracle's DATE also carries a time.	<code>opened_date</code> , <code>date_of_birth</code> , <code>maturity_date</code>
TIME	Time of day	Branch opening hours
TIMESTAMP	Date and time	<code>txn_date</code>
TIMESTAMP WITH TIME ZONE	An instant, plus a zone	Cross-border payments
INTERVAL	A duration	"90 days", "3 months"

The distinction that matters: a deposit **matures on a date**, so `DATE` is right. A withdrawal **happens at an instant**, so `TIMESTAMP` is right. Storing a transaction as `DATE` throws away the time and makes intraday analysis impossible.

The Oracle DATE trap. Oracle's `DATE` includes hours, minutes and seconds. PostgreSQL's `DATE` does not. Migrating an Oracle `DATE` column to a PostgreSQL `DATE` silently discards the time. This is one of the most common data-loss defects in heterogeneous migration, and Module 11 covers how to catch it.

2.5 Choosing types for Smart Saver

Column	Type	Reasoning
<code>deposit_id</code>	<code>INTEGER</code>	Fewer than 2 billion deposits will ever exist
<code>customer_id</code>	<code>INTEGER</code>	Must match <code>customer.customer_id</code> exactly, or the foreign key will not work
<code>branch_id</code>	<code>INTEGER</code>	Same reason
<code>receipt_number</code>	<code>VARCHAR(20)</code>	Printed on the customer's receipt, format may change
<code>principal_amount</code>	<code>NUMERIC(15,2)</code>	Money
<code>interest_rate</code>	<code>NUMERIC(5,2)</code>	Percentages such as 7.25, two decimals by product rule
<code>tenure_days</code>	<code>SMALLINT</code>	Maximum 3,650
<code>booked_date</code> , <code>maturity_date</code>	<code>DATE</code>	Calendar days
<code>auto_renew</code>	<code>CHAR(1)</code>	Exactly one character, 'Y' or 'N'
<code>status</code>	<code>VARCHAR(12)</code>	ACTIVE, MATURED, BROKEN, CLOSED

A foreign key column must have the same type as the column it references. If `customer.customer_id` is `INTEGER`, then `fixed_deposit.customer_id` must be `INTEGER` too. A mismatch either fails to create or, worse, creates and then silently prevents index use.

3. Constraints

A constraint is a rule the engine enforces on every write, from every application, forever. It is the mechanism that delivers the C in ACID.

Constraint	Guarantees
NOT NULL	A value must be supplied
UNIQUE	No two rows share this value
PRIMARY KEY	Unique and not null; identifies the row
FOREIGN KEY	The value exists in the referenced table
CHECK	The value satisfies a condition
DEFAULT	A value is supplied when the caller omits one

3.1 NOT NULL and DEFAULT

```
principal_amount NUMERIC(15,2) NOT NULL,
booked_date      DATE          DEFAULT CURRENT_DATE NOT NULL,
auto_renew       CHAR(1)       DEFAULT 'N' NOT NULL,
nominee_name     VARCHAR(60)    -- nullable on purpose
```

The question to ask for every column is: **can this fact be genuinely unknown for a row that is otherwise valid?** A deposit with no principal is not a deposit, so `NOT NULL`. A deposit with no nominee is entirely normal, so nullable.

`DEFAULT` and `NOT NULL` work well together. `booked_date DEFAULT CURRENT_DATE NOT NULL` means the caller may omit the date and still cannot end up with a `NULL`.

3.2 UNIQUE and PRIMARY KEY

```
CONSTRAINT pk_fixed_deposit PRIMARY KEY (deposit_id),
CONSTRAINT uq_fd_receipt   UNIQUE (receipt_number)
```

	PRIMARY KEY	UNIQUE
NULLs allowed	No	Yes, and usually more than one
Per table	Exactly one	As many as needed
Creates an index	Yes	Yes
Meaning	This is the row's identity	This value never repeats

`deposit_id` is the identity used by foreign keys and application code. `receipt_number` is a business value printed on the customer's receipt; it must never repeat, but it is not what the system uses to find the row.

Surrogate versus natural keys. `deposit_id` is a **surrogate** key: a meaningless number invented by the system. `receipt_number` is a **natural** key: it has business meaning. Surrogate keys are preferred as primary keys because business meaning changes. Receipt formats get redesigned; identities must not.

3.3 FOREIGN KEY and referential integrity

```
CONSTRAINT fk_fd_customer FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
```

This does two things. It rejects a deposit whose `customer_id` does not exist, and it rejects the deletion of a customer who still holds a deposit.

Referential action	Effect when the parent row is deleted
NO ACTION / RESTRICT	Refuse the delete. The correct default in banking.
ON DELETE CASCADE	Delete the child rows too
ON DELETE SET NULL	Set the child's foreign key to NULL; requires a nullable column

CASCADE is convenient and dangerous. Deleting one customer row would silently remove their accounts, transactions, loans and deposits. Banks do not delete customers; they close them. Module 4 examines this in detail.

3.4 CHECK constraints

```
CONSTRAINT ck_fd_principal CHECK (principal_amount >= 10000),
CONSTRAINT ck_fd_tenure CHECK (tenure_days BETWEEN 7 AND 3650),
CONSTRAINT ck_fd_rate CHECK (interest_rate > 0 AND interest_rate <= 15),
CONSTRAINT ck_fd_renew CHECK (auto_renew IN ('Y','N')),
CONSTRAINT ck_fd_status CHECK (status IN ('ACTIVE','MATURED','BROKEN','CLOSED')),
CONSTRAINT ck_fd_maturity CHECK (maturity_date > booked_date)
```

A **CHECK** can reference more than one column of the same row, which is how `ck_fd_maturity` works. It cannot reference other rows or other tables; that requires a trigger, covered in Module 10.

CHECK constraints are where product rules become enforceable. Smart Saver's 10,000 rupee minimum is now a property of the data, not a hope about the application.

3.5 Naming constraints

Always name them.

```
-- Unnamed
ERROR: new row for relation "fixed_deposit" violates check constraint "fixed_deposit_check1"

-- Named
ERROR: new row for relation "fixed_deposit" violates check constraint "ck_fd_principal"
```

The second message tells the support engineer at 2 a.m. exactly which product rule was broken. A consistent convention helps: `pk_`, `uq_`, `fk_`, `ck_` followed by the table and the column.

4. DDL

4.1 CREATE TABLE

```
CREATE TABLE fixed_deposit (
  deposit_id      INTEGER      NOT NULL,
  customer_id     INTEGER      NOT NULL,
  branch_id       INTEGER      NOT NULL,
  receipt_number  VARCHAR(20)  NOT NULL,
  principal_amount NUMERIC(15,2) NOT NULL,
  interest_rate   NUMERIC(5,2) NOT NULL,
  tenure_days     SMALLINT     NOT NULL,
  booked_date     DATE          DEFAULT CURRENT_DATE NOT NULL,
  maturity_date   DATE          NOT NULL,
  auto_renew      CHAR(1)       DEFAULT 'N' NOT NULL,
  nominee_name    VARCHAR(60),
  status          VARCHAR(12)   DEFAULT 'ACTIVE' NOT NULL,
  CONSTRAINT pk_fixed_deposit PRIMARY KEY (deposit_id),
  CONSTRAINT uq_fd_receipt  UNIQUE (receipt_number),
  CONSTRAINT fk_fd_customer FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
  CONSTRAINT fk_fd_branch   FOREIGN KEY (branch_id) REFERENCES branch(branch_id),
  CONSTRAINT ck_fd_principal CHECK (principal_amount >= 10000),
  CONSTRAINT ck_fd_tenure   CHECK (tenure_days BETWEEN 7 AND 3650),
  CONSTRAINT ck_fd_rate     CHECK (interest_rate > 0 AND interest_rate <= 15),
  CONSTRAINT ck_fd_renew    CHECK (auto_renew IN ('Y','N')),
  CONSTRAINT ck_fd_status   CHECK (status IN ('ACTIVE','MATURED','BROKEN','CLOSED')),
  CONSTRAINT ck_fd_maturity CHECK (maturity_date > booked_date)
);
```

4.2 ALTER TABLE

Requirements change, and a table that has been live for two years still has to change with them.

```
-- The product team adds nomination
ALTER TABLE fixed_deposit ADD COLUMN nominee_name VARCHAR(60);

-- The NRI variant needs three decimal places on the rate
ALTER TABLE fixed_deposit ALTER COLUMN interest_rate TYPE NUMERIC(6,3);

-- The minimum booking rises to 25,000 in April
ALTER TABLE fixed_deposit DROP CONSTRAINT ck_fd_principal;
ALTER TABLE fixed_deposit
  ADD CONSTRAINT ck_fd_principal CHECK (principal_amount >= 25000);

-- The pilot flag is no longer needed
ALTER TABLE fixed_deposit DROP COLUMN pilot_flag;
```

Two rules worth remembering:

- Adding a **NOT NULL** column to a table that already contains rows fails, unless you also give a **DEFAULT**. The engine has to put something in the existing rows.
- Adding a **CHECK** constraint to a table with existing rows fails if any existing row violates it. This is a feature: it tells you the data is already wrong.

4.3 Dialect differences

Task	PostgreSQL	Oracle	SQL Server	MySQL
Add a column	ADD COLUMN x INT	ADD (x NUMBER)	ADD x INT	ADD COLUMN x INT
Change a type	ALTER COLUMN x TYPE NUMERIC	MODIFY (x NUMBER)	ALTER COLUMN x DECIMAL	MODIFY COLUMN x DECIMAL
Rename a column	RENAME COLUMN a TO b	RENAME COLUMN a TO b	EXEC sp_rename 't.a', 'b', 'COLUMN'	CHANGE a b INT
Add a constraint	ADD CONSTRAINT ...	ADD CONSTRAINT ...	ADD CONSTRAINT ...	ADD CONSTRAINT ...

4.4 DROP, DELETE and TRUNCATE

	DELETE	TRUNCATE	DROP
Removes	Selected rows	All rows	The table itself
Category	DML	DDL	DDL
WHERE supported	Yes	No	No
Rollback	Yes	PostgreSQL and SQL Server yes; Oracle and MySQL no	Rarely
Fires triggers	Yes	No	No
Speed on 10 million rows	Minutes	Immediate	Immediate
Table remains	Yes	Yes	No
Resets identity counters	No	Usually yes	Not applicable

TRUNCATE deallocates the storage instead of removing rows one at a time, which is why it is instant and why it cannot fire row-level triggers or be filtered.

5. DML

5.1 INSERT

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES
(7001, 1001, 1, 'FD2026000001', 250000.00, 7.25, 365,
 DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

Always name the columns. The short form `INSERT INTO fixed_deposit VALUES (...)` depends on the physical column order and breaks the moment anyone adds a column. It is the `SELECT *` of the write path.

Multiple rows in one statement:

```
INSERT INTO fixed_deposit (deposit_id, customer_id, branch_id, receipt_number,
 principal_amount, interest_rate, tenure_days,
 booked_date, maturity_date, auto_renew, status)
VALUES
(7002, 1002, 2, 'FD2026000002', 100000.00, 6.90, 180,
 DATE '2026-03-02', DATE '2026-08-29', 'N', 'ACTIVE'),
(7003, 1003, 3, 'FD2026000003', 500000.00, 7.40, 730,
 DATE '2026-03-02', DATE '2028-03-01', 'Y', 'ACTIVE');
```

One statement inserting 1,000 rows is dramatically faster than 1,000 statements, because there is one round trip and one transaction instead of a thousand of each.

Loading from a query:

```
INSERT INTO fd_high_value (deposit_id, customer_id, principal_amount)
SELECT deposit_id, customer_id, principal_amount
FROM fixed_deposit
WHERE principal_amount >= 1000000;
```

`INSERT ... SELECT` is the workhorse of ETL and of the migration work in Module 11.

5.2 UPDATE

```
-- 1. SELECT first, with the WHERE clause you intend to use
SELECT deposit_id, status
FROM fixed_deposit
WHERE maturity_date <= CURRENT_DATE AND status = 'ACTIVE';

-- 2. Then the UPDATE, with exactly the same WHERE clause
UPDATE fixed_deposit
SET status = 'MATURED'
WHERE maturity_date <= CURRENT_DATE AND status = 'ACTIVE';
```

An `UPDATE` with no `WHERE` updates every row in the table. There is no confirmation prompt. The habit of writing the `WHERE` clause first, testing it with a `SELECT`, and only then converting it to an `UPDATE`, is what prevents this.

Relative updates.

```
UPDATE account SET balance = balance + 15000 WHERE account_id = 9046;  -- correct
UPDATE account SET balance = 40000 WHERE account_id = 9046;  -- wrong
```

The second form assumes nothing changed between your read and your write. If a UPI credit arrived in between, it is silently erased. This is the **lost update** problem, and Module 6 covers the locking that makes the relative form safe.

5.3 DELETE

```
SELECT * FROM fixed_deposit WHERE deposit_id = 7003;  -- look first
DELETE FROM fixed_deposit WHERE deposit_id = 7003;
```

In a bank, financial rows are almost never deleted. Regulation requires history, and reconciliation depends on it. The banking pattern is a **soft close**:

```
UPDATE fixed_deposit SET status = 'CLOSED' WHERE deposit_id = 7003;
```

The row stays, reporting can still explain last year's numbers, and the auditor can follow the trail.

6. SELECT

6.1 Clause order versus evaluation order

```
SELECT deposit_id, principal_amount -- written 1st, evaluated 5th
FROM   fixed_deposit               -- written 2nd, evaluated 1st
WHERE  status = 'ACTIVE'           -- written 3rd, evaluated 2nd
GROUP BY ...                       -- evaluated 3rd (Module 3)
HAVING ...                         -- evaluated 4th (Module 3)
ORDER BY maturity_date             -- evaluated 6th
LIMIT 10;                         -- evaluated 7th
```

This explains a rule that otherwise looks arbitrary: a column alias created in **SELECT** **cannot** be used in **WHERE**, because **WHERE** runs before **SELECT**. It **can** be used in **ORDER BY**, because **ORDER BY** runs after.

6.2 Projection

```
SELECT deposit_id,
       principal_amount,
       principal_amount * interest_rate / 100 AS annual_interest,
       UPPER(status) AS status
FROM   fixed_deposit;
```

Feature	Note
Expressions	Derive in the query rather than storing a value that can drift
AS alias	Names the output column; mandatory in practice on every expression
DISTINCT	Removes duplicate rows from the whole result, not from one column
Concatenation operator	Double pipe in PostgreSQL and Oracle; + in SQL Server; CONCAT() in MySQL

6.3 WHERE

Operator	Meaning	Example
= <>	Equality	status = 'ACTIVE'
< > <= >=	Ordering	principal_amount >= 100000
BETWEEN a AND b	Inclusive range	tenure_days BETWEEN 7 AND 365
IN (...)	Membership	branch_id IN (1,2,5)
LIKE	Pattern	receipt_number LIKE 'FD2026%'
IS NULL	Missing value	nominee_name IS NULL
AND OR NOT	Combination	See precedence below

The **BETWEEN** trap with timestamps.

```
-- Wrong: loses every transaction after midnight on 31 March
WHERE txn_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31'

-- Right: a half-open range
WHERE txn_date >= DATE '2026-01-01' AND txn_date < DATE '2026-04-01'
```

DATE '2026-03-31' is converted to 2026-03-31 00:00:00, so a transaction at 10:15 that day fails the test. An entire day of transactions disappears from the quarterly report, and nobody notices until reconciliation.

Operator precedence. AND binds more tightly than OR.

```
WHERE branch_id = 1 OR branch_id = 2 AND principal_amount > 500000
-- means: branch_id = 1 OR (branch_id = 2 AND principal_amount > 500000)

WHERE (branch_id = 1 OR branch_id = 2) AND principal_amount > 500000
-- almost certainly what was meant
```

Use brackets whenever AND and OR appear in the same predicate.

LIKE and indexes. LIKE 'FD2026%' can use an index, because the pattern is anchored at the start. LIKE '%SWIGGY%' cannot, and forces a full table scan. On a 40-million-row transaction table that is the difference between a report and an outage. Module 5 covers the alternatives.

6.4 NULL and three-valued logic

NULL means unknown. It is not zero, not an empty string, and not false.

```
SELECT NULL = NULL;           -- NULL, not true
SELECT 50000 + NULL;          -- NULL
SELECT * FROM fixed_deposit WHERE nominee_name = NULL; -- returns nothing
SELECT * FROM fixed_deposit WHERE nominee_name IS NULL; -- correct
```

A	B	A AND B	A OR B
TRUE	NULL	NULL	TRUE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

WHERE keeps only rows for which the condition evaluates to **TRUE**. Rows evaluating to NULL are dropped, silently. This produces a result that startles people:

```
SELECT COUNT(*) FROM fixed_deposit WHERE nominee_name = 'Priya Verma';
SELECT COUNT(*) FROM fixed_deposit WHERE nominee_name <> 'Priya Verma';
-- These two counts do not add up to COUNT(*) of the table,
-- because rows with a NULL nominee satisfy neither condition.
```

Handling NULL deliberately.

```
SELECT deposit_id,
       COALESCE(nominee_name, 'NOT NOMINATED') AS nominee,
       NULLIF(interest_rate, 0) AS rate,
       CASE WHEN nominee_name IS NULL THEN 'Follow up'
            ELSE 'Complete' END AS nomination_action
```

```
FROM fixed_deposit;
```

`COALESCE` returns its first non-null argument and is standard SQL. Oracle's `NVL` and SQL Server's `ISNULL` do the same thing but are not portable, so prefer `COALESCE`.

6.5 ORDER BY

```
SELECT deposit_id, customer_id, principal_amount, maturity_date
FROM fixed_deposit
WHERE status = 'ACTIVE'
ORDER BY maturity_date ASC, principal_amount DESC;
```

Without `ORDER BY`, the engine may return rows in any order at all, and that order can change between runs as the plan changes. **There is no natural order in a relational table.**

Feature	Note
ASC / DESC	Ascending is the default
Multiple keys	Applied left to right
NULLS FIRST / NULLS LAST	PostgreSQL and Oracle sort NULLs last by default in <code>ASC</code> ; SQL Server and MySQL sort them first
Sort by expression	<code>ORDER BY principal_amount * interest_rate DESC</code>
Sort by alias	Legal, because <code>ORDER BY</code> is evaluated after <code>SELECT</code>
Sort by position	<code>ORDER BY 3 DESC</code> is legal but fragile; avoid it in maintained code

6.6 Limiting rows

Engine	Syntax
PostgreSQL, MySQL	<code>SELECT ... ORDER BY x DESC LIMIT 10</code>
SQL Server	<code>SELECT TOP 10 ... ORDER BY x DESC</code>
Oracle 12c+, ANSI	<code>SELECT ... ORDER BY x DESC FETCH FIRST 10 ROWS ONLY</code>
Oracle 11g and earlier	<code>SELECT * FROM (SELECT ... ORDER BY x DESC) WHERE ROWNUM <= 10</code>

A "top 10" query without `ORDER BY` is meaningless. The engine returns whichever ten rows it encountered first, which may differ every run.

7. Built-in functions

Everything so far has moved rows around untouched. Functions transform the values on the way out. A bank uses them constantly: to mask a card number, to format a statement, to clean a vendor feed, to compute interest.

These are the ones you will reach for weekly. Every example runs against your seeded database.

7.1 Type conversion

Data arrives as the wrong type all the time. A vendor sends an amount as text. A report needs a date as a formatted string. Conversion is where most feed defects are caught or created.

CAST is the standard form. :: is the PostgreSQL shorthand. They do the same thing, and the standard form is the one that survives a migration.

```
SELECT CAST('250000' AS NUMERIC)      AS standard_form,
       '250000'::NUMERIC              AS postgres_shorthand,
       CAST(341244.80 AS INTEGER)     AS rounds_to_nearest;
```

standard_form	postgres_shorthand	rounds_to_nearest
250000	250000	341245

CAST to INTEGER rounds, it does not truncate. 341244.80 becomes 341245, not 341244. When you want the rupees and not the paise, use **TRUNC**, which is in 7.4. Getting this wrong on money is a real defect.

A cast fails loudly on bad input, which is what you want:

```
SELECT CAST('2,50,000' AS NUMERIC);
```

```
ERROR:  invalid input syntax for type numeric: "2,50,000"
```

Clean first, then cast. This is exactly the vendor feed problem in the exercises:

```
SELECT REPLACE('2,50,000', ',', ' '::NUMERIC AS cleaned;
```

cleaned
250000

Formatting with TO_CHAR

TO_CHAR turns a number or a date into text in a chosen shape. Statements, letters and regulatory files all need it.

```
SELECT TO_CHAR(balance, 'FM999999990.00')      AS amount_text,
       TO_CHAR(DATE '2026-01-15', 'DD-Mon-YYYY') AS statement_date,
       TO_CHAR(TIMESTAMP '2026-01-15 15:08:00',
                'DD/MM/YYYY HH24:MI')          AS txn_stamp
FROM   account WHERE account_id = 9003;
```

```
amount_text | statement_date | txn_stamp
-----+-----+-----
341244.80  | 15-Jan-2026   | 15/01/2026 15:08
```

Pattern	Means	Example output
FM	Strip the padding spaces	341244.80 rather than a padded column
9	A digit, blank if not needed	Leading blanks suppressed
0	A digit, zero if not needed	Forces 0.00 rather than .00
DD-Mon-YYYY	Day, short month, year	15-Jan-2026
HH24:MI	24-hour clock	15:08

TO_CHAR has no Indian lakh grouping. FM99,99,999.00 will not produce 3,41,244.80 correctly for every magnitude. Format money in the reporting layer, or store the rule in one function rather than repeating it.

Reading text back

```
SELECT TO_DATE('15-01-2026', 'DD-MM-YYYY') AS parsed_date,
       TO_NUMBER('341244.80', '999999990.00') AS parsed_amount;
```

```
parsed_date | parsed_amount
-----+-----
2026-01-15 | 341244.80
```

TO_DATE needs the pattern that matches the incoming text. Give it the wrong one and you get a wrong date rather than an error, which is the silent killer Module 11 returns to.

7.2 Handling NULL

Function	Returns	Banking use
COALESCE(a, b, c)	The first argument that is not NULL	A missing nominee shows as NOT NOMINATED
NULLIF(a, b)	NULL when the two are equal, otherwise a	Turn a zero divisor into NULL to avoid an error
GREATEST(a, b)	The largest argument	Charge the higher of a flat fee and a percentage
LEAST(a, b)	The smallest argument	Cap a cashback at a ceiling

```
SELECT receipt_number,
       COALESCE(nominee_name, 'NOT NOMINATED') AS nominee,
       GREATEST(principal_amount * 0.001, 100) AS processing_fee
```

```
FROM fixed_deposit
ORDER BY deposit_id
LIMIT 3;
```

`NULLIF` earns its place guarding division:

```
SELECT channel,
       SUM(amount) AS total,
       SUM(amount) / NULLIF(COUNT(*), 0) AS avg_safe
FROM account_txn
GROUP BY channel;
```

Without `NULLIF`, a channel with no rows raises `division by zero` and the whole report fails. With it, the row returns `NULL`, which a report can show as a dash.

7.3 String functions

The bank's text work is masking, cleaning and parsing.

Function	Does	Example
<code>LENGTH(s)</code>	Character count	<code>LENGTH('ICIC0000009003')</code> is 14
<code>LEFT(s, n)</code> and <code>RIGHT(s, n)</code>	First or last n characters	<code>RIGHT(account_number, 4)</code> gives 9003
<code>SUBSTRING(s FROM p FOR n)</code>	n characters from position p	Pull the branch part of an IFSC
<code>UPPER, LOWER, INITCAP</code>	Change case	<code>INITCAP('rohan iyer')</code> gives Rohan Iyer
<code>TRIM, LTRIM, RTRIM</code>	Strip spaces	Removes the trailing space in a branch name
<code>LPAD(s, n, c)</code> and <code>RPAD</code>	Pad to a width	Pad a reference number to a fixed length
<code>REPLACE(s, from, to)</code>	Swap every occurrence	Strip commas from a vendor amount
<code>POSITION(x IN s)</code>	Where x starts, 0 if absent	Find the at sign in an email
<code>SPLIT_PART(s, sep, n)</code>	The nth piece	Pull the merchant out of a narration

Masking, the way a contact centre needs it

```
SELECT account_number,
       'XXXXXXXX' || RIGHT(account_number, 4) AS masked
FROM account
WHERE account_id = 9003;
```

```
account_number | masked
-----+-----
ICIC0000009003 | XXXXXXXXX9003
```

The double pipe is the standard concatenation operator. Oracle uses it too. SQL Server uses `+`, and MySQL wants `CONCAT`. `CONCAT` also works in PostgreSQL, and unlike the operator it ignores `NULL` rather than swallowing the whole result.

```
SELECT first_name || ' ' || last_name AS with_operator,
       CONCAT(first_name, ' ', last_name) AS with_concat
FROM   customer WHERE customer_id = 1003;
```

```
with_operator | with_concat
-----+-----
Rohan Iyer   | Rohan Iyer
```

The operator returns NULL if any part is NULL. Concatenating a name with a NULL middle name gives NULL, not the name. `CONCAT` treats NULL as empty. This is a common cause of blank names on printed letters.

Parsing a narration

The transaction narrations in the seed look like `NEFT TFR MYNTRA`. The merchant is the third word.

```
SELECT narration,
       SPLIT_PART(narration, ' ', 1) AS channel_word,
       SPLIT_PART(narration, ' ', 3) AS merchant
FROM   account_txn
WHERE  txn_id = 700001;
```

```
narration      | channel_word | merchant
-----+-----+-----
NEFT TFR MYNTRA | NEFT         | MYNTRA
```

Cleaning, which Module 4 will need

```
SELECT LENGTH('Koramangala ') AS before_trim,
       LENGTH(TRIM('Koramangala ')) AS after_trim,
       UPPER(TRIM(' koramangala ')) AS normalised;
```

```
before_trim | after_trim | normalised
-----+-----+-----
12 | 11 | KORAMANGALA
```

That single trailing space is one of the four spellings of Koramangala you will meet in Module 4. A `TRIM` at load time would have prevented it.

7.4 Numeric functions

Function	Does	Note for money
<code>ROUND(n, d)</code>	Round to d decimals	The right choice for interest
<code>TRUNC(n, d)</code>	Cut off, never round up	Use when the bank must not over-credit
<code>CEIL(n)</code> and <code>FLOOR(n)</code>	Up or down to a whole number	Tenure in whole months
<code>ABS(n)</code>	Drop the sign	Compare a debit and a credit by size
<code>MOD(a, b)</code>	Remainder	Every nth account for a sample

Function	Does	Note for money
POWER(a, b)	a to the power b	Compound interest

Monthly simple interest on a savings account:

```
SELECT account_number,
       balance,
       interest_rate,
       ROUND(balance * interest_rate / 100 / 12, 2) AS monthly_interest
FROM   account
WHERE  account_id = 9003;
```

```
account_number | balance | interest_rate | monthly_interest
-----+-----+-----+-----
ICIC0000009003 | 341244.80 |          3.50 |          995.30
```

ROUND and TRUNC differ by a paisa here, and over millions of accounts a paisa is a reconciliation break:

```
SELECT ROUND(995.297, 2) AS rounded,
       TRUNC(995.297, 2) AS truncated;
```

```
rounded | truncated
-----+-----
995.30 |    995.29
```

Decide which one the product uses, and write it down. Interest credited to a customer is usually rounded. A fee charged to a customer is usually truncated in the customer's favour. Neither is a technical decision.

7.5 Date and time functions

Function	Returns
CURRENT_DATE	Today, as a date
NOW()	The current timestamp with time zone
EXTRACT(part FROM d)	One field: year, month, day, hour, dow
DATE_TRUNC(part, ts)	The timestamp flattened to the start of that part
AGE(d)	The interval between d and today, in years, months and days
d + INTEGER	A date that many days later
d1 - d2	The number of days between two dates

Grouping a statement by month, which Module 3 builds on:

```
SELECT DATE_TRUNC('month', txn_date) AS month_start,
       EXTRACT(YEAR FROM txn_date) AS yr,
       EXTRACT(MONTH FROM txn_date) AS mth
FROM   account_txn
```

```
WHERE txn_id = 700001;
```

month_start	yr	nth
2026-01-01 00:00:00	2026	1

Maturity arithmetic, the way a deposit works:

```
SELECT DATE '2026-03-01' AS booked,
       DATE '2026-03-01' + 365 AS matures,
       DATE '2027-03-01' - DATE '2026-03-01' AS days_between;
```

booked	matures	days_between
2026-03-01	2027-03-01	365

Customer age, for a product eligibility rule:

```
SELECT first_name, date_of_birth,
       EXTRACT(YEAR FROM AGE(date_of_birth)) AS age_years
FROM   customer
WHERE  customer_id = 1003;
```

The output depends on the day you run it, so it is not printed here. Rohan Iyer was born on 1982-03-07, and a senior citizen rule of 60 and above is the usual reason a bank computes this.

Never compute age as `(CURRENT_DATE - dob) / 365`. It drifts by a day every four years, and a customer turns 60 on the wrong date. `AGE` handles leap years.

7.6 CASE

`CASE` is not a function but it belongs here, because it is how a bank turns values into categories.

```
SELECT account_number, balance,
       CASE WHEN balance >= 100000 THEN 'PRIORITY'
            WHEN balance >= 50000 THEN 'GOLD'
            ELSE 'STANDARD'
       END AS segment
FROM   account
WHERE  account_id IN (9003, 9010)
ORDER BY account_id;
```

account_number	balance	segment
ICIC0000009003	341244.80	PRIORITY
ICIC0000009010	49313.33	STANDARD

Conditions are tested top to bottom and the first match wins, so order them from the most restrictive down. Without the `ELSE`, an unmatched row returns NULL rather than an error, which is a quiet way to lose rows in

a report.

7.7 The same job in three dialects

Modules 10 and 11 need this table. The function differs, the intent does not.

Job	PostgreSQL	Oracle	SQL Server
First non-null	COALESCE	NVL or COALESCE	ISNULL or COALESCE
Join two strings	Double pipe operator	Double pipe operator	The plus operator
Part of a string	SUBSTRING	SUBSTR	SUBSTRING
Today	CURRENT_DATE	SYSDATE	GETDATE()
Format a date	TO_CHAR	TO_CHAR	FORMAT or CONVERT
Text to number	CAST or double colon	TO_NUMBER	CAST or TRY_CAST
Add days	Add an integer	Add an integer	DATEADD
Days between	Subtract two dates	Subtract two dates	DATEDIFF
Cut off decimals	TRUNC	TRUNC	ROUND(n, d, 1)

****COALESCE and CAST are in the standard. NVL, ISNULL and SUBSTR are not.**** Prefer the standard spelling and your SQL survives the migration in Module 11 untouched.

8. The completed deliverable

The call centre's daily list, using every idea from this module:

```
SELECT  fd.receipt_number,
        c.first_name || ' ' || c.last_name      AS customer_name,
        c.phone,
        fd.principal_amount,
        fd.maturity_date,
        fd.auto_renew,
        COALESCE(fd.nominee_name, 'NOT NOMINATED') AS nominee
FROM    fixed_deposit fd
JOIN    customer c ON c.customer_id = fd.customer_id
WHERE   fd.status = 'ACTIVE'
AND     fd.maturity_date >= CURRENT_DATE
AND     fd.maturity_date < CURRENT_DATE + 7
ORDER   BY fd.maturity_date, fd.principal_amount DESC;
```

Idea from this module	Where it appears
Named columns rather than *	The <code>SELECT</code> list
Expression with an alias	Concatenating first and last name into <code>customer_name</code>
NULL handling	<code>COALESCE(fd.nominee_name, ...)</code>
Half-open date range	<code>>= CURRENT_DATE AND < CURRENT_DATE + 7</code>
Multi-key sort	<code>ORDER BY maturity_date, principal_amount DESC</code>
Referential integrity paying off	The join to <code>customer</code> cannot produce an orphan

Terms introduced in this module

Term	Meaning
DDL, DML, DCL, TCL	Definition, manipulation, control and transaction sub-languages of SQL
Implicit commit	DDL silently committing an open transaction, on Oracle and MySQL
Surrogate key	A meaningless system-generated identifier used as the primary key
Natural key	A business value that is unique, such as a receipt number
Referential action	What happens to children when a parent row is deleted
Three-valued logic	TRUE, FALSE and NULL, and the way NULL propagates
Lost update	A concurrent change silently overwritten by an absolute assignment
Half-open range	<code>>= start AND < end</code> , the safe way to filter timestamps

Where this module leads

You learned	It becomes
<code>SELECT</code> from one table	Module 3, joining several
<code>CHECK</code> and <code>FOREIGN KEY</code>	Module 4, designing the whole schema

You learned	It becomes
<code>LIKE '%x%'</code> cannot use an index	Module 5, indexing and execution plans
Relative updates and lost updates	Module 6, locking and isolation
<code>INSERT ... SELECT</code>	Module 11, bulk data migration
The <code>fixed_deposit</code> table	Modules 3, 5, 6 and the capstone